The Journal of Open Source Software

Joule Profiler: A phase-based energy measurement tool

Jérémy Woirhaye[1,2*], **François Gibier**[1,2*], **and Romain Rouvoy**[1,2]

1 Inria, France **2** University of Lille, France * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Mojtaba Barzegari

Reviewers:

- @shrishar
- @StewMH
- @ygrange

Submitted: 03 May 2026
Published: unpublished

Summary

Joule Profiler is a lightweight Linux command-line tool for measuring a program's energy consumption with minimal instrumentation overhead. It enables users to break execution into user-defined phases (e.g., data loading, computation) and to attribute energy use to each. The tool detects phase triggers from program output and automatically queries sources like Intel RAPL (CPUs) or NVML (GPUs) to report system-wide energy consumption.

Statement of need

Energy use in computing is a growing concern across research and industry. Software running in clouds, data centres, and edge devices contributes significantly to global energy consumption. Improving efficiency requires tools that measure energy during execution. Hardware counters provided by modern CPUs and GPUs (e.g., Intel RAPL) make software-based energy measurement possible without external devices.

Researchers and developers need simple tools to measure the energy use of code segments without complex infrastructure. Joule Profiler addresses this with phase-based profiling that integrates easily into workflows.

State of the field

Existing tools like PowerAPI (Fieni et al., 2024), Alumet (Raffin & Trystram, 2025), Scaphandre (Hubblo contributors, 2021), and EnergiBridge (Sallou et al., 2023) monitor energy using these counters, often focusing on distributed and system-level observability. JouleIt (powerapi-ng contributors, 2022), which inspired this work, demonstrated a light wrapper approach but lacked phase decomposition, GPU support, and modularity.

These solutions are suited for system-level monitoring, not fine-grained analysis of program phases. Joule Profiler, in contrast, is designed for lightweight, single-invocation use, enabling energy attribution to specific phases within programs.

Phase-based profiling

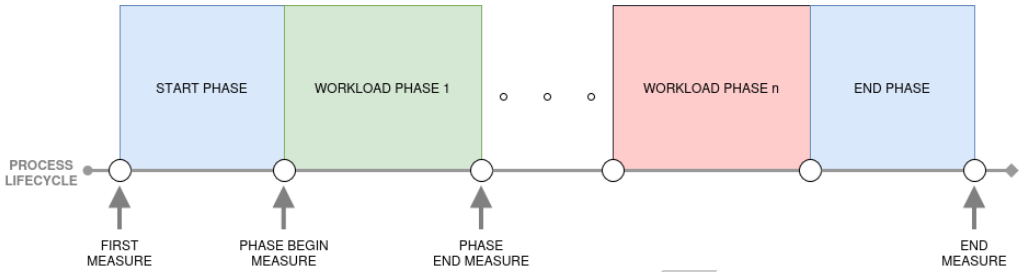

Figure 1: Process lifecycle illustrating sequential phases

Traditional energy measurement provides either total energy or periodic power readings, leaving unclear which code regions are most energy-intensive. Joule Profiler enables phase-based profiling, letting users decompose execution into logical phases with minimal code changes by watching standard output for phase markers.

Joule Profiler scans standard output for user-defined patterns to detect phase boundaries. Developers can insert print statements at important program points if needed, enabling phase identification without intrusive instrumentation.

When a phase marker is detected, Joule Profiler records energy counter values at that boundary. After execution, it computes per-phase energy by subtracting these values.

Software design

Joule Profiler's modular design separates measurement logic from hardware specifics. It accesses energy and performance metrics using `perf_event` (Linux Kernel Documentation, n.d.) (or powercap as a fallback) for RAPL, and NVML (NVIDIA, n.d.) for NVIDIA GPUs. The tool can correlate energy with performance counters, supporting extension and maintenance.

The tool uses a layered structure: the core detects phases and aggregates metrics; sources run asynchronously for parallel data collection; the CLI manages user interaction; and hardware backends are abstracted for easy integration of new sources.

Validity of the energy measurement

To validate its measurements, Joule Profiler was compared with reference tools perf (Linux perf developers, 2024) and Alumet (Raffin & Trystram, 2025), both using RAPL counters but different strategies. This checks whether Joule Profiler introduces measurement bias.

Three scenarios were tested: (1) parallel runs of Joule Profiler and perf (with CPU load) or Alumet (with GPU load) alongside a sleep command, ensuring identical hardware activity and measurement noise; (2) Sequential execution of Joule Profiler, perf, and Alumet with workload pinned to a single CPU core, to compare overhead and variability; and (3) A custom workload with periodic output tokens tested phase detection precision.

Experiments used Grid'5000 nodes: Chirop (Intel Xeon, RAPL, 512 GiB RAM) and Chifflot (Nvidia Tesla V100, NVML, 192 GiB RAM). Energy was measured from RAPL (PACKAGE, DRAM) and NVML (GPU). `perf_event` was used for access. Hyper-threading was disabled, and the CPU frequency governor was set to performance to reduce variability.

Total energy comparison

Parallel execution

We performed 4,000 measurements to achieve a statistical power of 80% and applied a *Two One-Sided Tests* (TOST) procedure with an equivalence margin of 0.1% of the reference tool's mean to assess statistical equivalence.

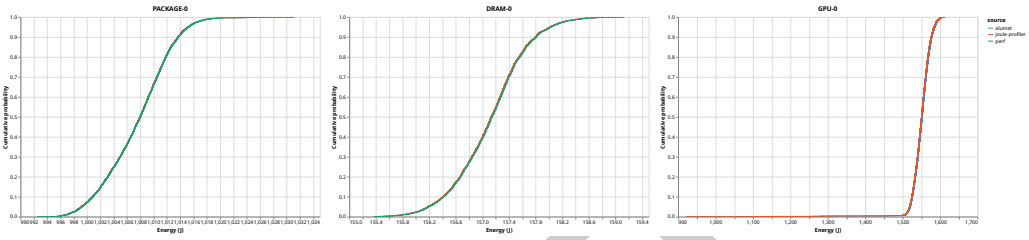

Figure 2: *Empirical Cumulative Distribution Function* (ECDF) of energy measurements (J) across RAPL domains (DRAM, PACKAGE) comparing perf and Joule Profiler, and GPU comparing Alumet and Joule Profiler.

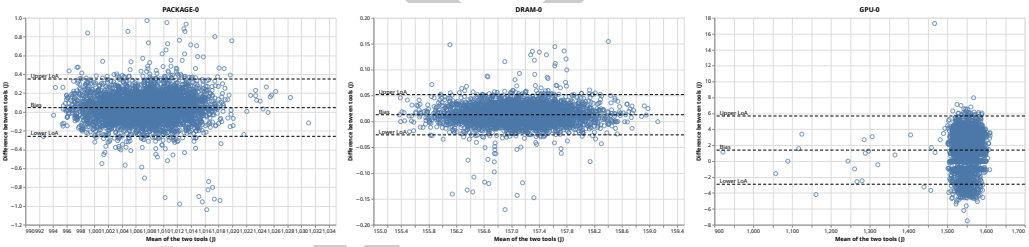

Figure 3: Bland–Altman analysis of energy measurements (J) across RAPL domains (DRAM, PACKAGE) comparing 'perf' and Joule Profiler, and GPU comparing Alumet and Joule Profiler.

Figure 2 and Figure 3 show close agreement between Joule Profiler and the reference tools. For `DRAM-0`, the bias is 0.013 J with 96.8% of measurements within the Limits of Agreement (LoA). For `PACKAGE-0`, the bias is 0.046 J with 95.8% within LoA, though higher variability is observed at high energy values, consistent with known RAPL noise at the package level. For `GPU-0`, the bias is 1.39 J with 94.5% within LoA, reflecting the higher natural variability of GPU power sampling (coefficient of variation ~1.95% for both tools). The Pearson correlation between Joule Profiler and perf exceeded 99.9% for both RAPL domains, and reached 99.5% against Alumet for GPU. The TOST null hypotheses of non-equivalence were rejected for all domains, confirming that Joule Profiler does not introduce a significant measurement bias.

Sequential execution

A sequential execution (2,000 runs) was used to compare the tool's overhead and variability. All tools produced nearly identical distributions, with $<0.1\%$ difference (RAPL) and $<0.5\%$ (GPU), indicating minimal overhead.

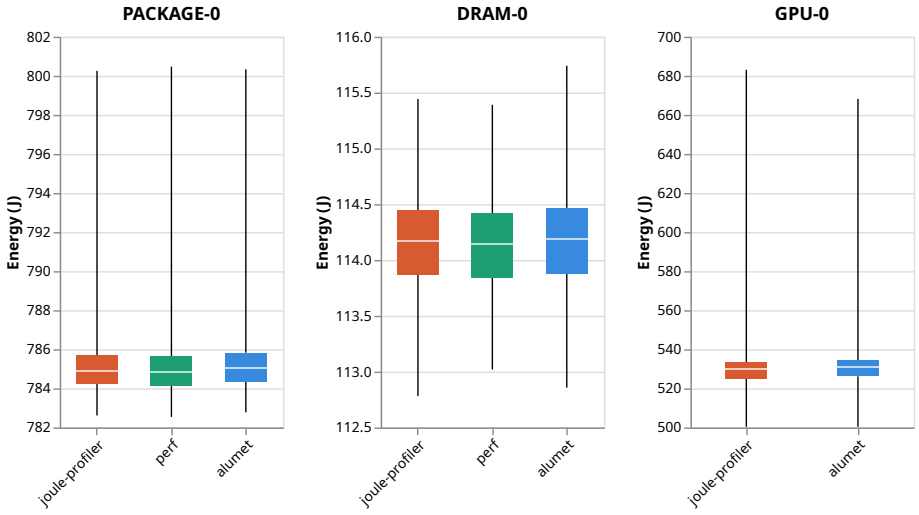

Figure 4: Energy distribution (J) across RAPL domains (DRAM, PACKAGE) and GPU comparing perf, Alumet, and Joule Profiler.

Figure 4 presents the energy distributions of `perf`, Joule Profiler, and Alumet across sequential runs for RAPL domains and the GPU. As for the parallel scenario, all tools report nearly identical values, with differences of less than 0.1% for RAPL domains and 0.5% for GPU. The sequential execution results show that Joule Profiler does not introduce a significant overhead compared to Alumet and perf.

Phase attribution precision

To evaluate the temporal accuracy of output-based phase detection, we used a custom program that printed periodic tokens at frequencies from 100 Hz to 1,000 Hz, comparing the timestamp at print time with the timestamp at which Joule Profiler detected each token. This was repeated 40 times at each frequency, with 10,000 measures per iteration, to achieve 80% statistical power.

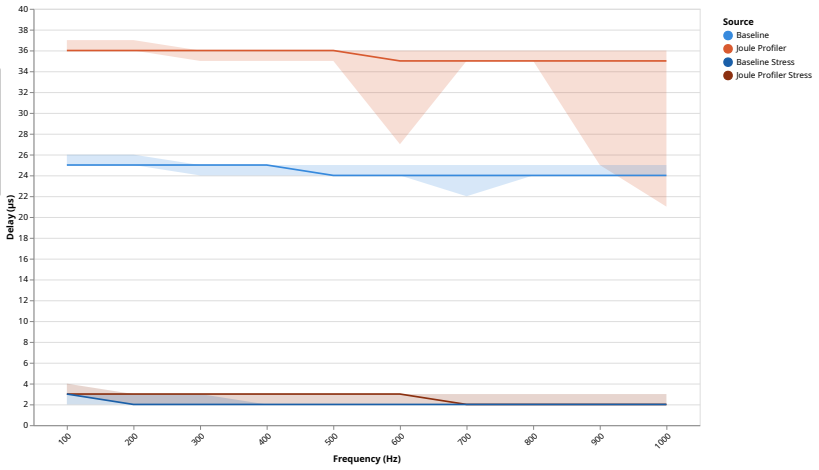

Figure 5: Median, first and last quartiles delay between phase detection time and real phase start.

Figure 5 shows that the baseline median detection delay is approximately 25 µs and remains stable across all frequencies. Joule Profiler introduces an additional median delay of 11 µs, with

a coefficient of variation increasing from 23% below 800 Hz to 28% at 1,000 Hz. Under CPU load (stress-ng), the baseline delay drops to 2 µs, and the Joule Profiler to 3 µs, confirming that idle-state latency is the primary source of delay. These results confirm that output-based instrumentation is viable for workloads with phase durations exceeding 1 ms, consistent with the RAPL counter refresh rate of 1,000 Hz.

Research impact statement

Joule Profiler was developed at Inria and the University of Lille, supported by the France 2030 program under grant agreement `ANR-23-PECL-0003` (CARECloud project of the PEPR CLOUD research program). It benchmarks Function-as-a-Service workloads, isolating costs by phase. The work is also part of the PULSE project with Qarnot Computing, focusing on energy-aware software engineering for heterogeneous environments.

All validation experiments used the Grid'5000/SLICES-FR testbed, a shared French research infrastructure. Joule Profiler is intentionally compatible with its hardware and workflows.

Joule Profiler is open-source (MIT) at https://github.com/joule-profiler/joule-profiler, with versioned releases and documentation.

AI Usage Disclosure

This submission used generative AI tools only during early project stages.

Tool identification.
The authors used Claude Sonnet 4.5 (Anthropic) as a generative AI assistant during the project bootstrap phase.

Scope of assistance.
AI assisted with repository structure, initial boilerplate, and early organisational guidance.

Human verification and oversight.
All AI outputs were reviewed and validated by the authors, who made all key decisions and ensured compliance with standards.

The authors take full responsibility for the accuracy, originality, and integrity of the submitted work.

Acknowledgements

This work received support from the France 2030 program under grant agreement No. `ANR-23-PECL-0003` (PEPR CLOUD CARECloud), and from the Inria–Qarnot PULSE project (https://defi-pulse.github.io/). Experiments were carried out using the Grid'5000 testbed, supported by a scientific interest group hosted by Inria and including CNRS, RENATER, and several Universities (see https://www.grid5000.fr).

References

Fieni, G., Acero, D. R., Rust, P., & Rouvoy, R. (2024). PowerAPI: A Python framework for building software-defined power meters. *Journal of Open Source Software*, *9*(98), 6670. https://doi.org/10.21105/joss.06670

Hubblo contributors. (2021). *Scaphandre: Energy Consumption Metrics Agent for Linux*. https://github.com/hubblo-org/scaphandre

Linux Kernel Documentation. (n.d.). *Perf events and tool security*. https://docs.kernel.org/admin-guide/perf-security.html.

Linux perf developers. (2024). *Perf: Linux profiling with performance counters*. https://perfwiki.github.io/main/

NVIDIA. (n.d.). *NVIDIA management library (NVML) documentation*. https://docs.nvidia.com/deploy/nvml-api/.

powerapi-ng contributors. (2022). *JouleIt: A bash script for measuring energy consumption on Linux*. https://github.com/powerapi-ng/jouleit

Raffin, G., & Trystram, D. (2025). Dissecting the software-based measurement of CPU energy consumption: A comparative analysis. *IEEE Transactions on Parallel and Distributed Systems*, *36*(1), 96–107. https://doi.org/10.1109/TPDS.2024.3492336

Sallou, J., Cruz, L., & Durieux, T. (2023). *EnergiBridge: Empowering software sustainability through cross-platform energy measurement*. https://doi.org/10.48550/arXiv.2312.13897